Learning With AI — Topic 2

IoT Networking — ESP32 WiFi to Cloud Pipeline

Garvin Patel | patelg5@mymail.nku.edu

IoT Smart Parking System | NKU Spring 2026

The Complete IoT Pipeline

> *"Data doesn't magically appear on a dashboard — it travels through a carefully connected chain of technologies."*

```
HC-SR04 Sensor
      ↓  measures distance every 3 seconds
XIAO ESP32C6
      ↓  WiFi.begin() + HTTPClient POST
Python Flask API
      ↓  pymongo upsert
MongoDB Atlas (Cloud)
      ↓  fetch() every 3 seconds
React.js Dashboard
      ↓
Any device, anywhere
```

Step 1 — ESP32 WiFi Connection

How WiFi.begin() Actually Works

```
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}

Serial.println(WiFi.localIP()); // e.g. 172.20.10.3
```

- ESP32 sends credentials to router
- Router assigns a unique **IP address**
- Every device on same network gets `192.168.x.x` range

WiFi Debugging — Status Codes

What I Learned From Real Errors

Status Code	Meaning	My Fix
`0`	Idle	Waiting to connect
`3`	Connected ✅	Success!
`6`	Wrong password	Simplified hotspot password
`-1`	Connection refused	Wrong IP or different network

Real error I faced:

```
Flask response: -1
```

Step 2 — HTTP POST Request

ESP32 Sends Data to Flask

```
HTTPClient http;
http.begin("http://192.168.50.68:5001/update");
http.addHeader("Content-Type", "application/json");
http.setTimeout(5000);

String payload = "{\"spot_id\":\"A1\","
                 "\"distance_cm\":" + String(distance) + "}";

int responseCode = http.POST(payload);
// 200 = success, -1 = connection refused
http.end();
```

- **JSON** is the standard format for sending structured data

Step 3 — Flask REST API

Receiving and Processing Sensor Data

```python
@app.route('/update', methods=['POST'])
def update_spot():
    data = request.get_json()
    spot_id = data.get('spot_id', 'A1')
    distance = data.get('distance_cm', 0)
    status = "OCCUPIED" if distance < 50 else "AVAILABLE"

    spots_collection.update_one(
        {"spot_id": spot_id},
        {"$set": {"status": status, "distance_cm": distance}},
        upsert=True  # update if exists, insert if new
    )
    return jsonify({"status": status})
```

Step 4 — MongoDB Atlas

Cloud Database Storage

```
MONGO_URI = "mongodb+srv://user:pass@cluster.mongodb.net/"
client = MongoClient(MONGO_URI, tlsAllowInvalidCertificates=True)
db = client["smart_parking"]
spots_collection = db["spots"]
readings_collection = db["readings"]
```

Collection	What It Stores
`spots`	Current status of each spot (updated)
`readings`	Full history of every sensor reading

Step 5 — React Live Dashboard

Fetching Data Every 3 Seconds

```
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch("http://192.168.50.68:5001/status");
    const data = await res.json();
    setSpots(data.spots);
    setStats({ total: data.total, available: data.available,
               occupied: data.occupied });
  };

  fetchData();                              // fetch immediately
  const interval = setInterval(fetchData, 3000); // then every 3s
  return () => clearInterval(interval); // cleanup on unmount
}, []);
```

CORS — Allowing Cross-Origin Requests

Why React Couldn't Talk to Flask

Without CORS, browser blocks React from fetching Flask:

```
Access to fetch blocked by CORS policy
```

Fix in Flask:

```
from flask_cors import CORS
app = Flask(__name__)
CORS(app)  # allows any origin to access Flask
```

- **CORS** = Cross-Origin Resource Sharing

HTTP vs HTTPS Problem

Chrome Breaking the ESP32 Dashboard

What happened:

- ESP32 hosts webpage at `http://172.20.10.3`
- Chrome automatically redirected to `https://172.20.10.3`
- ESP32 only supports HTTP, not HTTPS
- Result: Dashboard wouldn't load

Fix:

- Explicitly type `http://` in Chrome address bar
- Chrome respects it when you type the full protocol

What I Learned — Summary

Concept	What I Understood
`WiFi.begin()`	Connects ESP32 to network, gets IP address
IP addresses	Devices must be on same subnet to communicate
HTTP POST	Sends JSON data from ESP32 to Flask
`@app.route`	Defines URL endpoints in Flask
`upsert=True`	Updates MongoDB without duplicating records
CORS	Allows React to fetch from Flask API
`useEffect`	Runs code when React component loads
`setInterval`	Repeatedly fetches data every 3 seconds
HTTP vs HTTPS	ESP32 only supports unencrypted HTTP

Key Takeaway

> *"Building an IoT system taught me that networking is not magic — it is a chain of precise connections where every link must match. IP addresses, ports, protocols, and CORS policies all have to align perfectly. AI helped me understand each error message and fix each broken link in the chain."*

Garvin Patel | Smart Parking IoT System | NKU Spring 2026